Introduction to LLVM

CMPT 379: Compilers
Instructor: Anoop Sarkar
anoopsarkar.github.io/compilers-class

Setting up

```
static llvm::Module *TheModule;

static llvm::LLVMContext TheContext;

static llvm::IRBuilder<> Builder(TheContext);
```

This global variable contains all the generated code.

The calls to Builder will sometimes use TheContext.

This is the method used to construct the LLVM intermediate code (IR).

Make sure your yacc actions incrementally generate instructions in the right order

To print out the LLVM output:

```
TheModule->print(llvm::errs(), nullptr);
```

Types in LLVM

decafType is not an LLVM data type. Your yacc program defines it for all the types in a Decaf program

```
llvm::Type *getLLVMType(decafType ty) {
	switch (ty) {
		case voidTy: return Builder.getVoidTy();
		case intTy: return Builder.getInt32Ty();
		case boolTy: return Builder.getInt1Ty();
		case stringTy: return llvm::PointerType::getUnqual(Builder.getInt8Ty());
		default: throw runtime_error("unknown type in getType call");
	}
}
```

Constants in LLVM

```
llvm::Constant *getZeroInit(decafType ty) {
    switch (ty) {
        case intTy: return Builder.getInt32(0);
        case boolTy: return Builder.getInt1(0);
        default: throw runtime_error("unknown type");
    }
}

llvm::Value *StringConstAST::Codegen() {
    const char *s = StringConst.c_str();
    llvm::Value *GS = Builder.CreateGlobalString(s, "globalstring");
    return GS;
    // e.g. can be used in Builder.CreateCall(print_string, GS);
}
```

Local Variables in LLVM

```
llvm::AllocaInst *defineVariable(
            llvm::Type *llvmTy,
            string ident)
{
   llvm::AllocaInst* Alloca =
                        Builder.CreateAlloca(llvmTy, 0, ident.c_str());
   syms.enter_symtbl(ident, Alloca);
   return Alloca;
}
```

This is the symbol table you need to keep information about each identifier. We store an AllocaInst* for each variable.

Using the Variable:

```
   llvm::AllocaInst *V = (AllocaInst *)syms.access_symtbl(Name);
   if (V == NULL) throw runtime_error("could not find variable: " + Name);
   return Builder.CreateLoad(V->getAllocatedType(), V, Name.c_str());
```

Assignment and checking types

```
a = b
```

We need to check if type of lvalue a is the same as type of rvalue b

```
assign: T_ID T_ASSIGN expr
```

Name

```
llvm::AllocaInst *Alloca
= (llvm::AllocaInst *)
syms.access_symtbl(Name);
```

rvalue

```
const llvm::PointerType *ptrTy =
llvm::PointerType::getUnqual(getTy);
```

```
if (ptrTy == Alloca->getType()) {
  Builder.CreateStore(rvalue, Alloca);
}
```

Declaring a Function in LLVM

```
// initalize return type
llvm::Type *returnTy = getLLVMType(ReturnType);
std::vector<llvm::Type *> args;
// args := initialize the vector of argument types
 llvm::Function *func = llvm::Function::Create(
       llvm::FunctionType::get(returnTy, args, false),
       llvm::Function::ExternalLinkage,
       Name,
       TheModule
    );
 syms.enter_symtbl(Name, func);
```

This is the symbol table you need to keep information about each identifier

Promoting Types in LLVM

- What if the variable is of type i1 (boolean)
- But the function only takes i32 (int)
- We must promote the type i1 to i32
- LLVM can do that for you using the ZExt instruction

```
llvm::Value *promo =
        Builder.CreateZExt(*i, Builder.getInt32Ty(), "zexttmp");
```

Basic Blocks in LLVM

```
// Create a new basic block which contains a sequence of LLVM instructions
llvm::BasicBlock *BB =
        llvm::BasicBlock::Create(
                TheContext,
                "entry",
                func);
// insert into symbol table
syms.enter_symtbl(string("entry"), BB);
// All subsequent calls to IRBuilder will place instructions in this location
Builder.SetInsertPoint(BB);
```

Function parameters

When you generate code for a method declaration do the following:

1. Create a new symbol table for local variables
2. Create a `BasicBlock`, let's say `BB`
3. Set insertion point for instructions `Builder.SetInsertPoint(BB)`
4. Add the arguments to the function as allocated on the stack (next slide)
5. You can check if a function has a `return` statement by checking if the value of `BB->getTerminator()` is `NULL`.

Function parameters

```
func foo(x int) int {
        x = 1;
}
```

For `Function* func` iterate through the function arguments and allocate them into the stack.

```
for (auto &Arg : func->args()) {
  llvm::AllocaInst *Alloca =
       CreateEntryBlockAlloca(func, Arg.getName());
  // Store the initial value into the alloca
  Builder.CreateStore(&Arg, Alloca);
  // Add to symbol table
  syms.enter_symtbl(Arg.getName(), Alloca);
}
```

Useful Tricks in LLVM

- Finding the current function you are in:

```
llvm::Function *func = Builder.GetInsertBlock()->getParent();
```

- External function

```
llvm::Function::Create(
        llvm::FunctionType::get(returnTy, args, false),
        llvm::Function::ExternalLinkage,
        Name,
        TheModule);
```

Modulus in LLVM

- Use `CreateSRem()` for signed operators in Decaf
- LLVM uses the C/C++ style modulus
- So `-4%3 == -1`

Backward Function Declarations

Backwards Declarations

```
extern func print_int(int) void;
package Test {
  func main() int {
    test(10, 13);
  }
  func test(a int, b int) void {
    print_int(a);
    print_int(b);
  }
}
```

Iterate through the list of function signatures and insert them into the symbol table.

Control Flow in LLVM

“Backpatching” in LLVM

- Inside `IfStmt->Codegen`:
  - Set up a new symbol table for code locations
  - Create a new BasicBlock called `iftrue` (see slide 9)
  - Create a new BasicBlock called `iffalse`
  - Create a new BasicBlock called `end`
  - Subsequent code generation anywhere else can insert code into these code locations
  - Can be used for break, continue, short-circuits, etc.

“Backpatching” in LLVM

Setting up the branching between Basic Blocks:

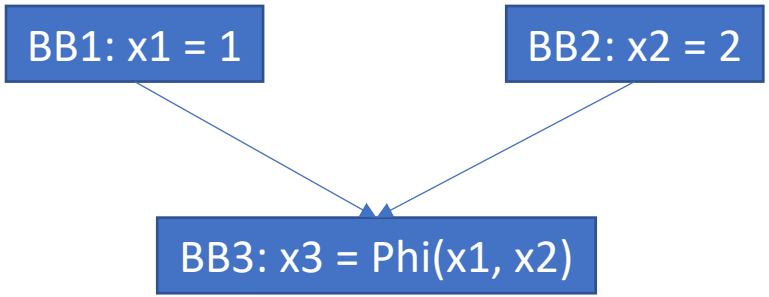
```
// val contains the Expr value for the conditional
Builder.CreateCondBr(val, IfTrueBB, EndBB);
Builder.SetInsertPoint(IfTrueBB);
IfTrueBlock->Codegen();
```

After the `IfStmt` we continue with the end Basic Block:

```
Builder.CreateBr(EndBB);
// pop the symbol table after IfStmt Codegen is done
Builder.SetInsertPoint(EndBB);
```

Static Single Assignment in LLVM

- For normal control flow using CreateBr and CreateCondBr no need for Phi functions
- LLVM produces the Phi functions automatically using algorithms we will study in class



Loops and local variables

- In loops (while/for) we create a declare basic block and declare the variables in the loop block:

```
// On first entry, generate var declarations:
llvm::BasicBlock *DeclBB = llvm::BasicBlock::Create(
    TheContext,
    "decl",
    func
);
Builder.SetInsertPoint(DeclBB);
val = Body->getVars()->Codegen();
Builder.CreateBr(BodyBB);
```

Loops and local variables

- We can avoid defining the variables in the loop basic block each time by skipping the DeclBB but then we must reset the values at the end of the loop.

```
    val = Body->getVars()->ResetValues();

// ResetValues:
    llvm::AllocaInst *Alloca = (llvm::AllocaInst *)
        syms.access_symtbl(local_variable->getName());
    Builder.CreateStore(
       getZeroInit(local_variable->getType()),
       Alloca
    );
```

Short-circuit of boolean expressions

- For short circuit of boolean expressions you have to write the PHI function yourself

```
llvm::PHINode *val =
      Builder.CreatePHI(type, 2, "phival");
// type is an LLVM::Type
val->addIncoming(L, CurBB);
val->addIncoming(opval, OpValBB);
// CurBB and OpValBB are the two basic blocks that are
incoming blocks for the PHI function
```

Short-circuit of boolean expressions

```
package sckt {
	func main() int {
		var a, b, c bool;
		a = true; b = false;
		c = a || b;
	}
}
```

Short-circuit of boolean expressions

```
; ModuleID = 'sckt'
source_filename = "DecafComp"
define i32 @main() {
func:
  ; removed all variable init code
  store i1 true, i1* %a
  store i1 false, i1* %b
  %a1 = load i1, i1* %a
  br i1 %a1, label %skctend, label %noskct
```

```
noskct: ; preds = %func
  %b2 = load i1, i1* %b
  %ortmp = or i1 %a1, %b2
  br label %skctend
```

```
skctend: ; preds = %noskct, %func
  %phival =
    phi i1 [ %a1, %func ], [ %ortmp, %noskct ]
  store i1 %phival, i1* %c
  ret i32 0
}
```

```
c = a || b;
```

if a is True then
do not evaluate b

Q: Why is CreatePHI using OpValBB not NoSkctBB aka nosckt: ?

Short-circuit of boolean expressions

```
; ModuleID = 'sckt'
source_filename = "DecafComp"
define i32 @main() {
func:
  ; removed all variable init code
  store i1 true, i1* %a
  store i1 false, i1* %b
  %a1 = load i1, i1* %a
  br i1 %a1, label %skctend, label %noskct
```

llvm::Value *L = LHS->Codegen();

```
c = a || b;
```

if a is True then do not evaluate b

```
noskct: ; preds = %func
  %b2 = load i1, i1* %b
  %ortmp = or i1 %a1, %b2
  br label %skctend
```

llvm::Value *opval = Builder.CreateOr(L, R, "ortmp");
llvm::BasicBlock *OpValBB = Builder.GetInsertBlock();

```
skctend: ; preds = %noskct, %func
  %phival =
    phi i1 [ %a1, %func ], [ %ortmp, %noskct ]
  store i1 %phival, i1* %c
  ret i32 0
```

llvm::PHINode *val =
  Builder.CreatePHI(L->getType(), 2, "phival");
val->addIncoming(L, CurBB); / * func */
val->addIncoming(opval, OpValBB); / * noskct */

GetElementPointer (GEP)

Dealing with non-scalar types

- Consider this C code:

```
struct munger_struct {
  int f1;
  int f2;
};
void munge(struct munger_struct *P) {
  P[0].f1 = P[1].f1 + P[2].f2;
}
struct munger_struct Array[3];
munge(Array);
```

```
struct munger_struct {
  int f1;
  int f2;
};
void munge(struct munger_struct *P) {
  P[0].f1 = P[1].f1 + P[2].f2;
}
```

```
define void @munge(ptr %P) {
entry:
  %tmp = getelementptr %struct.munger_struct, ptr %P, i32 1, i32 0
  %tmp1 = load i32, ptr %tmp
  %tmp2 = getelementptr %struct.munger_struct, ptr %P, i32 2, i32 1
  %tmp3 = load i32, ptr %tmp2
  %tmp4 = add i32 %tmp3, %tmp1
  %tmp5 = getelementptr %struct.munger_struct, ptr %P, i32 0, i32 0
  store i32 %tmp4, ptr %tmp5
  ret void
}
```

Checking types for a CreateLoad

- Sometimes you need to load from stack (AllocaInst) or from field variable (global variables in Decaf)

```
llvm::Type *inferType(llvm::Value *val) {
    if (llvm::isa<llvm::GlobalVariable>(val)) {
        llvm::GlobalVariable *gv = llvm::cast<llvm::GlobalVariable>(val);
        return gv->getValueType();
    } else if (llvm::isa<llvm::AllocaInst>(val)) {
        llvm::AllocaInst *alloca = llvm::cast<llvm::AllocaInst>(val);
        return alloca->getAllocatedType();
    } else {
        val->getType()->print(llvm::outs());
        return val->getType();
    }
}
```

Why is the extra 0 index required?

- This question arises most often when the GEP instruction is applied to a global variable which is always a pointer type.

```
%MyStruct = external global { ptr, i32 }
...
%idx = getelementptr { ptr, i32 }, ptr %MyStruct, i64 0, i32 1
```

- The type of %MyStruct is *not* { ptr, i32 } but rather ptr. That is, %MyStruct is a pointer (to a structure), not a structure itself.
- The first index, i64 0 is required to step over the global variable %MyStruct.
- The second index, i32 1 selects the second field of the structure (the i32).

What is dereferenced by GEP?

- The GetElementPtr instruction dereferences nothing. That is, it doesn't access memory in any way.
- That's what the Load and Store instructions are for.
- GEP is only involved in the computation of addresses.

```
@MyVar = external global { i32, [40 x i32 ] }
...
%idx = getelementptr { i32, [40 x i32] }, ptr @MyVar, i64 0, i32 1, i64 17
```

- GEP instruction can index through the global variable, into the second field of the structure and access the 18th i32 in the array there.

What does inbounds mean?

- The inbounds keyword is designed to describe low-level pointer arithmetic overflow conditions
- It does not refer to any array indexing rules.
- With the inbounds keyword, the result value of the GEP is poison if the address is outside the actual underlying allocated object and not the address one-past-the-end.
- Performing a load or a store requires an address of allocated and sufficiently aligned memory.
- But the GEP itself is only concerned with computing addresses.

Global variables and Arrays in LLVM

Checking types for a CreateLoad

- Sometimes you need to load from stack (AllocaInst) or from field variable (global variables in Decaf)

```
llvm::Type *inferType(llvm::Value *val) {
    if (llvm::isa<llvm::GlobalVariable>(val)) {
        llvm::GlobalVariable *gv = llvm::cast<llvm::GlobalVariable>(val);
        return gv->getValueType();
    } else if (llvm::isa<llvm::AllocaInst>(val)) {
        llvm::AllocaInst *alloca = llvm::cast<llvm::AllocaInst>(val);
        return alloca->getAllocatedType();
    } else {
        val->getType()->print(llvm::outs());
        return val->getType();
    }
}
```

Array types

- For int and bool types in Decaf we have field declarations that can be arrays over that type. To get the array type use `ArrayType`

```
llvm::ArrayType *getArrayType(decafType ty, int size) {
 switch (ty) {
    case intTy: return llvm::ArrayType::get(Builder.getInt32Ty(), size);
    case boolTy: return llvm::ArrayType::get(Builder.getInt1Ty(), size);
    default: throw runtime_error("unknown type in getArrayType call");
 }
}
```

Checking if `llvm::Value*` is an array

- Since we are storing `llvm::Value*` in the symbol table, we sometimes have to check if it is an array. In Decaf arrays are always field declarations (global variables).

```
bool isGlobalArrayType(llvm::Value *val) {
    if (NULL == val) {
        return false;
    }
    if (llvm::isa<llvm::GlobalVariable>(val)) {
        llvm::AllocaInst *Alloca = (llvm::AllocaInst *)val;
        return (Builder.getInt32(0) != Alloca->getArraySize());
    } else {
        return false;
    }
}
```

Getting a value from an array

- Let's assume we have the `i32` value of the index into the array `llvm::GlobalVariable *array` stored as `llvm::Value *index`

```
llvm::Value *zero =
llvm::ConstantInt::get(llvm::Type::getInt32Ty(TheContext), 0);

llvm::Value *arrayloc = Builder.CreateInBoundsGEP(
        array->getValueType(),
        array,
        {zero, index},
        "arrayloc"
    );
llvm::Type *ptrTy = array->getValueType();
llvm::ArrayType *arrTy = llvm::dyn_cast<llvm::ArrayType>(ptrTy);
llvm::Type *elemTy = arrTy->getElementType();
return Builder.CreateLoad(elemTy, arrayloc, "arrayval");
```

Assigning a new value into an array location

- This is similar to reading a value from an array assuming we have an rvalue defined as `llvm::Value* rvalue`

```
llvm::Value *lvalue = Builder.CreateInBoundsGEP(
        array->getValueType(),
        array,
        {zero, index},
        "arrayloc"
    );
const llvm::ArrayType *arrTy =
   static_cast<llvm::ArrayType *>(array->getValueType());
return Builder.CreateStore(rvalue, lvalue);
```